Student Performance Management System

PYTHON PROGRAM REFERENCE & IMPLEMENTATION GUIDE

Overview

This document contains the source code and reference architecture for a terminal-based **Student Performance Management System** implemented in Python. The system provides an interactive loop for managing student records dynamically using dictionary structures embedded inside an in-memory collection array list.

System Architecture & Capabilities

1. Data Storage

Utilizes dynamic standard dictionary components nested within a master list structure to process fields asynchronously.

2. Automation Engine

Automatically computes conditional validation thresholds (Pass/Fail criteria at 40 marks).

3. Lookup Matrix

Case-insensitive searching mechanics matching linear string queries against cached entries.

4. Aggregate Analytics

Mathematical calculation iteration mapping scores to evaluate continuous performance statistics.

Python Implementation Source Code

Below is the complete, production-ready Python codebase featuring a structured user command menu interface.

```
# Initialize an empty list to store student data structures
students = []

while True:
    print("\n===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====")
    print("1. Add Student")
    print("2. View Students")
    print("3. Search Student")
    print("4. Delete Student")
    print("5. Show Average Marks")
    print("6. Exit")

    choice = input("Enter your choice: ")
```

```
    if choice == "1":
        sid = input("Enter Student ID: ")
        name = input("Enter Student Name: ")
        dept = input("Enter Department: ")
        marks = float(input("Enter Marks: "))

        # Automated Grade Classification Rule
        if marks >= 40:
            result = "PASS"
        else:
            result = "FAIL"

        student = {
            "ID": sid,
            "Name": name,
            "Department": dept,
            "Marks": marks,
            "Result": result
        }

        students.append(student)
        print("Student Added Successfully!")

    elif choice == "2":
        if len(students) == 0:
            print("No student records found.")
        else:
            print("\n===== STUDENT RECORDS =====")
            for student in students:
                print("----------------------------")
                print("ID:", student["ID"])
                print("Name:", student["Name"])
                print("Department:", student["Department"])
                print("Marks:", student["Marks"])
                print("Result:", student["Result"])

    elif choice == "3":
        search_name = input("Enter Student Name to Search: ")
        found = False

        for student in students:
            if student["Name"].lower() == search_name.lower():
                print("\nStudent Found")
                print("ID:", student["ID"])
                print("Name:", student["Name"])
                print("Department:", student["Department"])
                print("Marks:", student["Marks"])
                print("Result:", student["Result"])
                found = True

        if not found:
            print("Student Not Found!")
```

```
    elif choice == "4":
        delete_name = input("Enter Student Name to Delete: ")
        found = False

        for student in students:
            if student["Name"].lower() == delete_name.lower():
                students.remove(student)
                print("Student Deleted Successfully!")
                found = True
                break

        if not found:
            print("Student Not Found!")

    elif choice == "5":
        if len(students) == 0:
            print("No student records available.")
        else:
            total = 0
            for student in students:
                total += student["Marks"]

            average = total / len(students)
            print("Average Marks =", round(average, 2))

    elif choice == "6":
        print("Thank You!")
        break

    else:
        print("Invalid Choice!")
```

End of Document • Designed for Python 3.x Implementations